Optimisation des Requêtes SQL



fait par Maati Rima Baya / Bettaieb Eya / Jabali Samar

Sommaire



- Introduction ---------- 01
- Fonctionnement d'une requête SQL ---------- 02
- Techniques d'optimisation ---------- 03
- Analyse et monitoring ---------- 04
- Conclusion ---------- 05

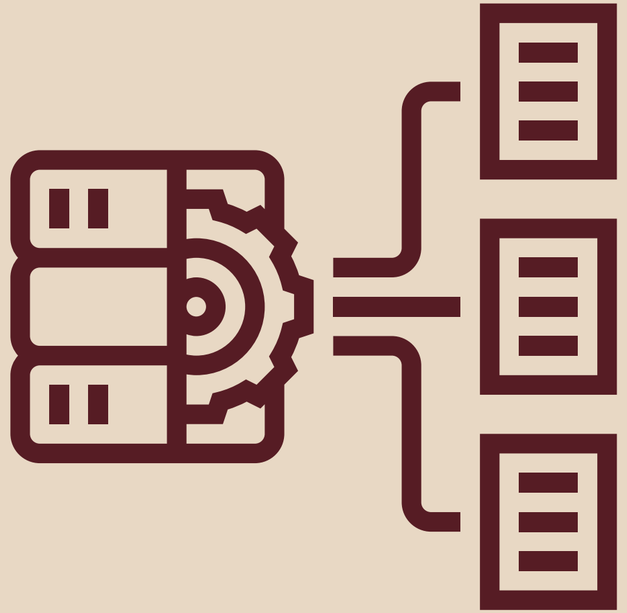
Introduction

- Avec le développement rapide des technologies de l'information, les bases de données sont devenues un élément essentiel dans le fonctionnement des systèmes informatiques. Elles sont utilisées dans de nombreux domaines tels que les sites web, les applications mobiles, les systèmes bancaires, les entreprises ou encore les plateformes de commerce en ligne. Ces systèmes manipulent chaque jour de très grandes quantités de données, parfois des millions ou même des milliards d'enregistrements.



- Pour interagir avec ces données, on utilise généralement le langage SQL (Structured Query Language), qui permet d'effectuer différentes opérations comme la recherche, l'insertion, la modification ou la suppression de données. Cependant, lorsque les bases de données deviennent volumineuses et que les requêtes sont nombreuses ou complexes, certaines requêtes SQL peuvent prendre beaucoup de temps à s'exécuter. Cela peut ralentir les applications, diminuer les performances du système et affecter l'expérience des utilisateurs.

- C'est dans ce contexte qu'intervient **l'optimisation des requêtes SQL**. L'optimisation consiste à améliorer la manière dont les requêtes sont écrites et exécutées afin de réduire leur temps d'exécution et d'utiliser les ressources du système de façon plus efficace.



Fonctionnement d’une requête SQL

- 1. Lecture du plan d’exécution:

  Comprendre le fonctionnement d’une requête SQL est important pour pouvoir l’optimiser. Lorsqu’une requête est envoyée au système de gestion de base de données (SGBD), elle passe par plusieurs étapes avant de retourner le résultat.

Comment le moteur SQL traite les requêtes?

- Lorsqu'une requête SQL est exécutée, le moteur de la base de données suit généralement les étapes suivantes :

1. Analyse syntaxique (Parsing):

- Le SGBD vérifie si la requête est correcte selon les règles du langage SQL.

2. Vérification des objets:

- Le système vérifie l'existence des tables et des colonnes mentionnées dans la requête ainsi que les droits d'accès de l'utilisateur.

3. Génération du plan d'exécution:

- Le SGBD détermine la meilleure stratégie pour accéder aux données (lecture de table, utilisation d'index, ordre des opérations, etc.).

4. Exécution de la requête:

- Le moteur exécute le plan choisi, récupère les données demandées et renvoie le résultat à l'utilisateur.

⚙ Le rôle du plan d'exécution

Le plan d'exécution représente la manière dont le moteur SQL décide d'exécuter une requête. Il indique notamment :

- l'ordre d'accès aux tables
- les méthodes de recherche des données
- l'utilisation ou non des index
- les opérations de tri ou de regroupement

Le plan d'exécution est généré automatiquement par l'optimiseur du SGBD. Celui-ci analyse différentes stratégies possibles et choisit celle qui semble la plus efficace en fonction de plusieurs facteurs comme :

- la taille des tables
- la présence d'index
- les statistiques sur les données
- la complexité de la requête

Comprendre ce plan permet d'identifier les parties d'une requête qui consomment le plus de ressources et qui pourraient être améliorées.

les techniques d'optimisation

1.Les Bonnes pratiques pour l’écriture des requêtes:

A-Évitez le SELECT *:

Utiliser SELECT * peut sembler pratique, mais cela entraîne des requêtes inefficaces. La base de données lit et transfère plus de données que nécessaire, ce qui augmente l’utilisation de la mémoire et ralentit les performances. Il est préférable de sélectionner uniquement les colonnes utiles afin d’optimiser les performances, réduire la charge du serveur et garder un code plus clair et maintenable.

requete non optimisée

```sql
SELECT *
FROM clients
WHERE ville = 'Paris';
```

requete optimisée

```sql
SELECT id, nom, age
FROM clients
WHERE ville = 'Paris';
```

B- Optimisez les clauses WHERE:

- La clause WHERE est fondamentale pour filtrer les données et optimiser les performances des requêtes SQL. Pour maximiser l'efficacité, il est recommandé de placer les conditions les plus restrictives au début du filtrage et d'éviter d'appliquer des fonctions sur les colonnes, ce qui permet à la base de données d'utiliser correctement les index.

- De plus, il est conseillé d'utiliser des conditions simples et précises dans la clause WHERE, comme les comparaisons directes (=, >, <, BETWEEN). L'utilisation excessive d'opérateurs comme LIKE avec des caractères génériques au début (%mot) ou de calculs sur les colonnes peut ralentir la recherche, car la base de données doit analyser un grand nombre de lignes.

```sql
SELECT *
FROM commandes
WHERE MONTH(date_commande) = 5;
```

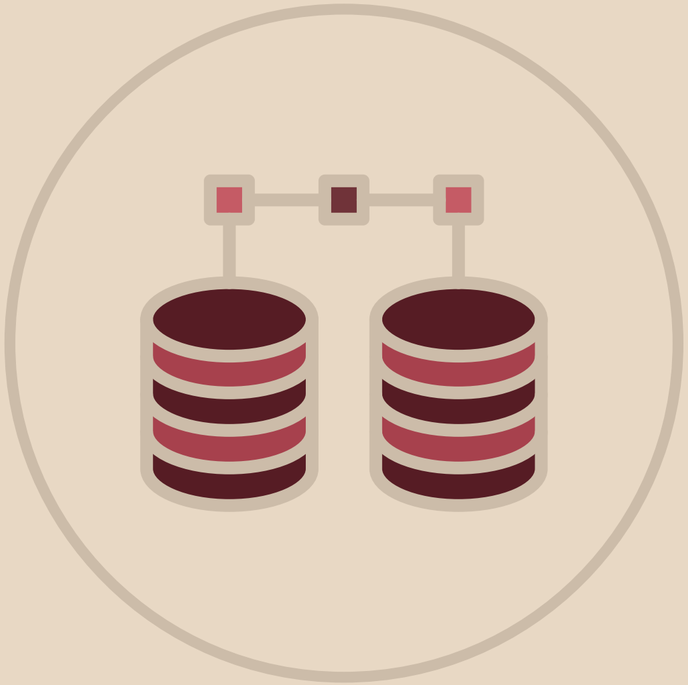
La fonction MONTH() est appliquée sur la colonne date_commande, ce qui empêche l'utilisation de l'index. La base de données doit donc parcourir toute la table pour trouver les résultats.

```sql
SELECT *
FROM commandes
WHERE date_commande BETWEEN '2024-05-01' AND '2024-05-31';
```

La condition utilise directement la colonne date_commande, ce qui permet à la base de données d'utiliser un index et d'accélérer la recherche.

C- **Éviter la recherche de données redondantes ou inutiles:**

- Pour optimiser les performances des requêtes SQL, il est crucial de limiter le nombre de lignes récupérées, en plus de sélectionner uniquement les colonnes nécessaires. La clause LIMIT permet d'éviter le chargement excessif de données, ce qui est idéal pour la validation, l'inspection et les tests. Cependant, elle ne doit pas être utilisée dans les modèles automatisés qui nécessitent l'intégralité des données.





Cette requête peut retourner un très grand nombre de lignes, même si seulement quelques résultats sont nécessaires.



Cette requête retourne seulement 10 lignes, ce qui permet de tester ou d'analyser rapidement les données sans charger toute la table.

2- Indexation:

A- Définition et rôle des index:

- Un index est une structure utilisée dans une base de données pour accélérer la recherche des informations dans une table.
- Il fonctionne comme l'index d'un livre : au lieu de lire toutes les pages pour trouver une information, on utilise l'index pour aller directement à la bonne page.
- Dans SQL, les index permettent donc de réduire le temps d'exécution des requêtes, surtout lorsque la table contient beaucoup de données.

B- types des index:

1 - Index simple:

- Il est créé sur une seule colonne d'une table.
- Il est utile lorsque les recherches se font souvent sur cette colonne.

```sql
CREATE INDEX idx_ville ON clients(ville);
```



```sql
SELECT nom
FROM clients
WHERE ville =
'Paris';
```

- La base de données peut utiliser l'index idx_ville pour aller directement aux lignes où ville = 'Paris' au lieu de parcourir toute la table.

1 - Index composite:

- Il est créé sur plusieurs colonnes à la fois.
- Il est utilisé lorsque les requêtes filtrent les données avec plusieurs conditions.

```sql
CREATE INDEX idx_client_date
ON commandes(id_client, date_commande);
```

```sql
SELECT *
FROM commandes
WHERE id_client = 101
  AND date_commande BETWEEN '2024-01-01' AND '2024-01-31';
```

- La base de données peut utiliser l'index pour trouver rapidement les commandes d'un client sur une période spécifique.

B- types des index:

1 - Index unique:

- Il garantit que les valeurs d'une colonne sont uniques dans la table.
- Il empêche donc l'insertion de valeurs dupliquées.

```sql
CREATE UNIQUE INDEX idx_code_produit
ON produits(code_produit);
```

```sql
INSERT INTO
produits(code_produit,
nom_produit)
VALUES ('P123', 'Stylo');
```

- Si P123 existe déjà, la base refuse l'insertion.

C - Quand créer un index et quand l'éviter?:

On crée généralement un index sur :

- les colonnes utilisées dans WHERE
- les colonnes utilisées dans JOIN
- les colonnes utilisées dans ORDER BY

Mais il faut éviter de créer trop d'index :

- sur les petites tables
- sur les colonnes rarement utilisées

➤ car cela peut augmenter l'espace de stockage



D- L’impact des index sur les performances:

- Les index améliorent surtout la vitesse des requêtes SELECT, car ils permettent de trouver les données plus rapidement.
- Cependant, ils peuvent ralentir les opérations INSERT, UPDATE et DELETE, car chaque modification dans la table nécessite aussi une mise à jour des index.

➤ **L’indexation est donc une technique essentielle pour améliorer les performances des bases de données, mais elle doit être utilisée de manière équilibrée pour éviter un impact négatif sur les opérations de modification des donn**

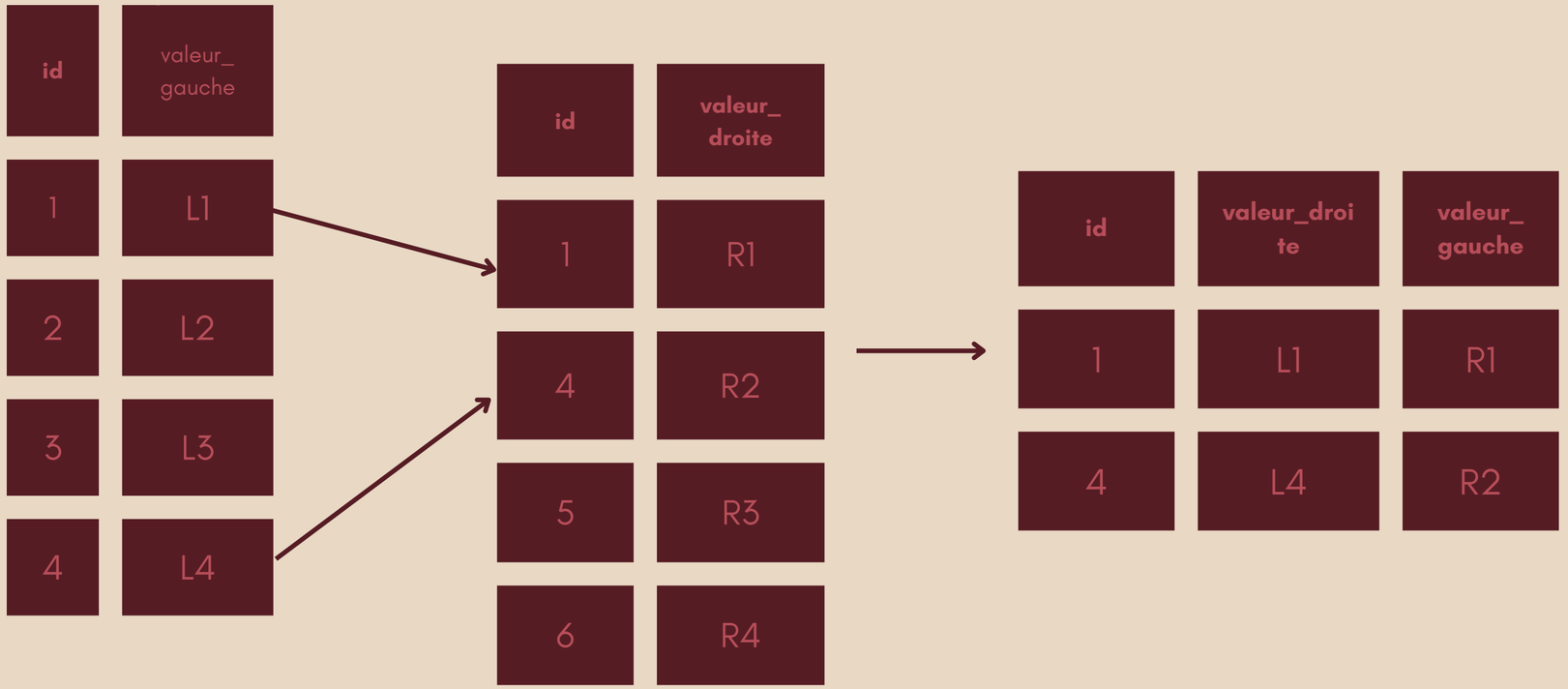
3- optimisation des jointures:

- Le langage SQL utilise les jointures (JOIN), qui permettent de relier les lignes de deux tables ou plus à partir d'une colonne commune
- Toutefois, lorsque les tables contiennent un grand volume de données ou que plusieurs jointures sont utilisées, les performances peuvent être affectées. **L'optimisation des jointures** devient alors essentielle pour améliorer les performances des requêtes SQL.

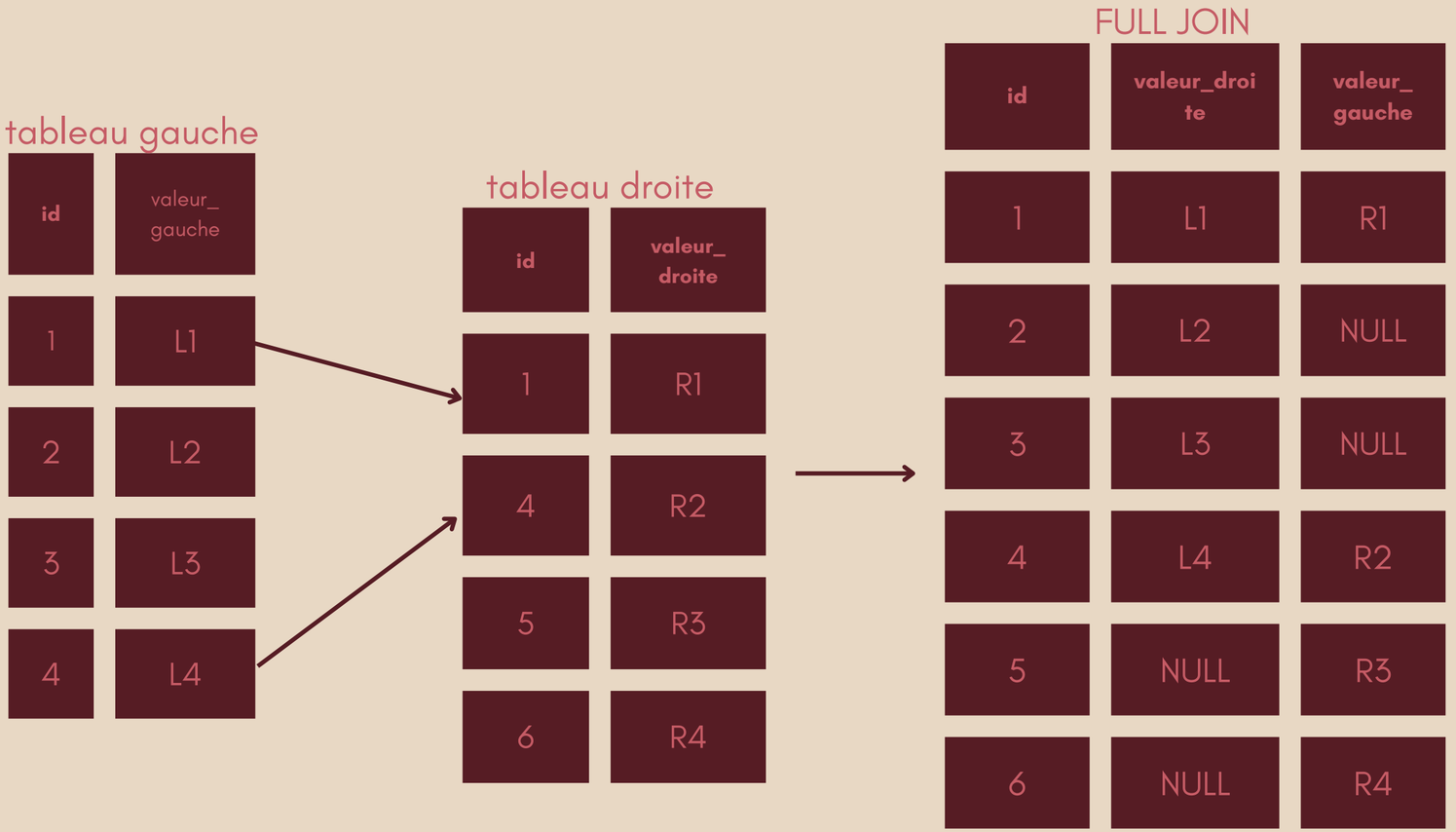
A- Choisir le bon type de jointure :

- Il existe plusieurs types de jointures : INNER JOIN, LEFT JOIN, RIGHT JOIN et FULL JOIN. Le choix du type de jointure influence directement la quantité de données traitées.

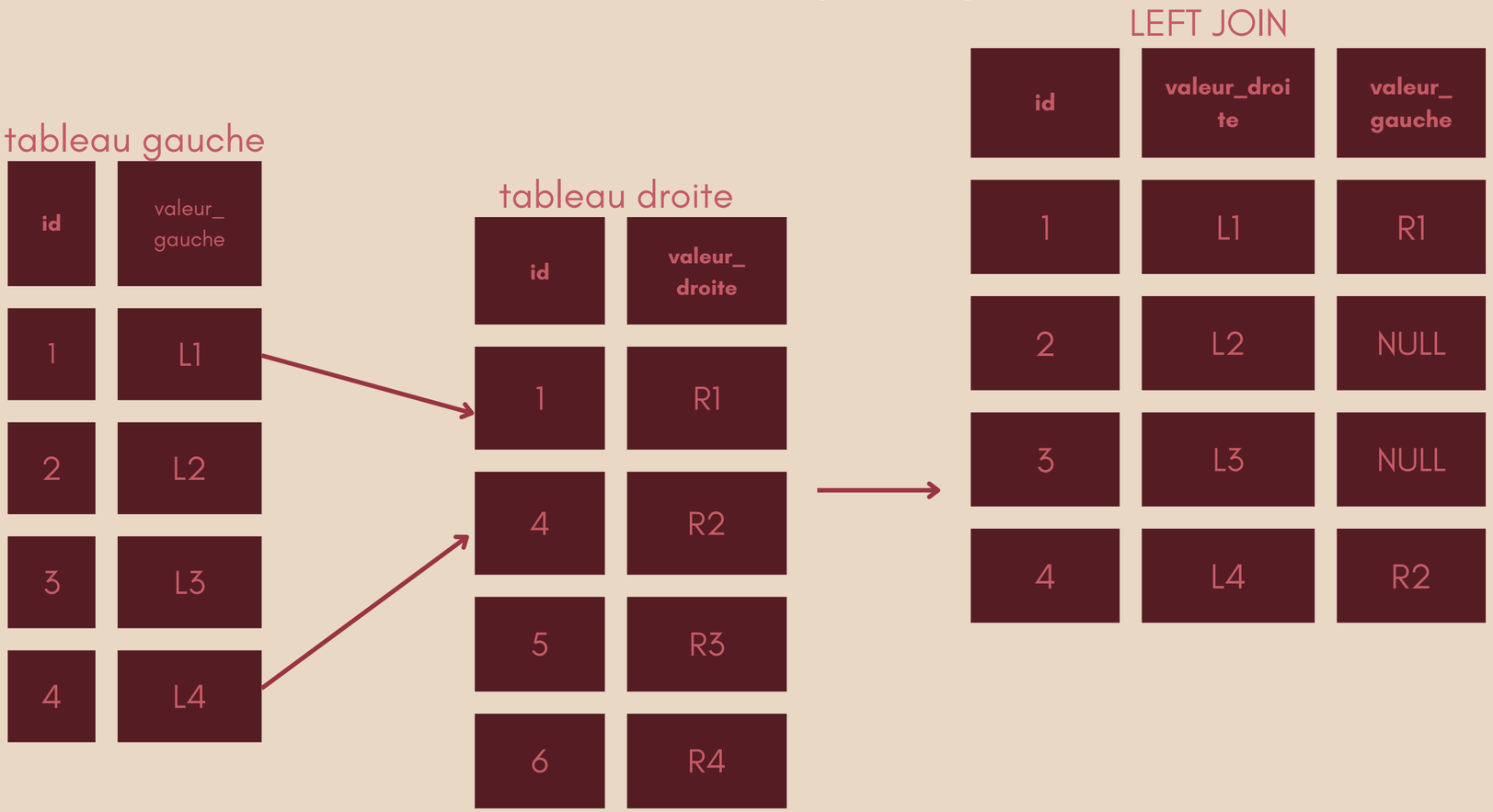
- **Une jointure interne(inner join):**
  ne renvoie que les lignes correspondant aux deux tableaux. Si un enregistrement existe dans un tableau mais pas dans l'autre, il sera exclu du résultat. C'est généralement le type de jointure le plus performant, car il réduit le volume de données.



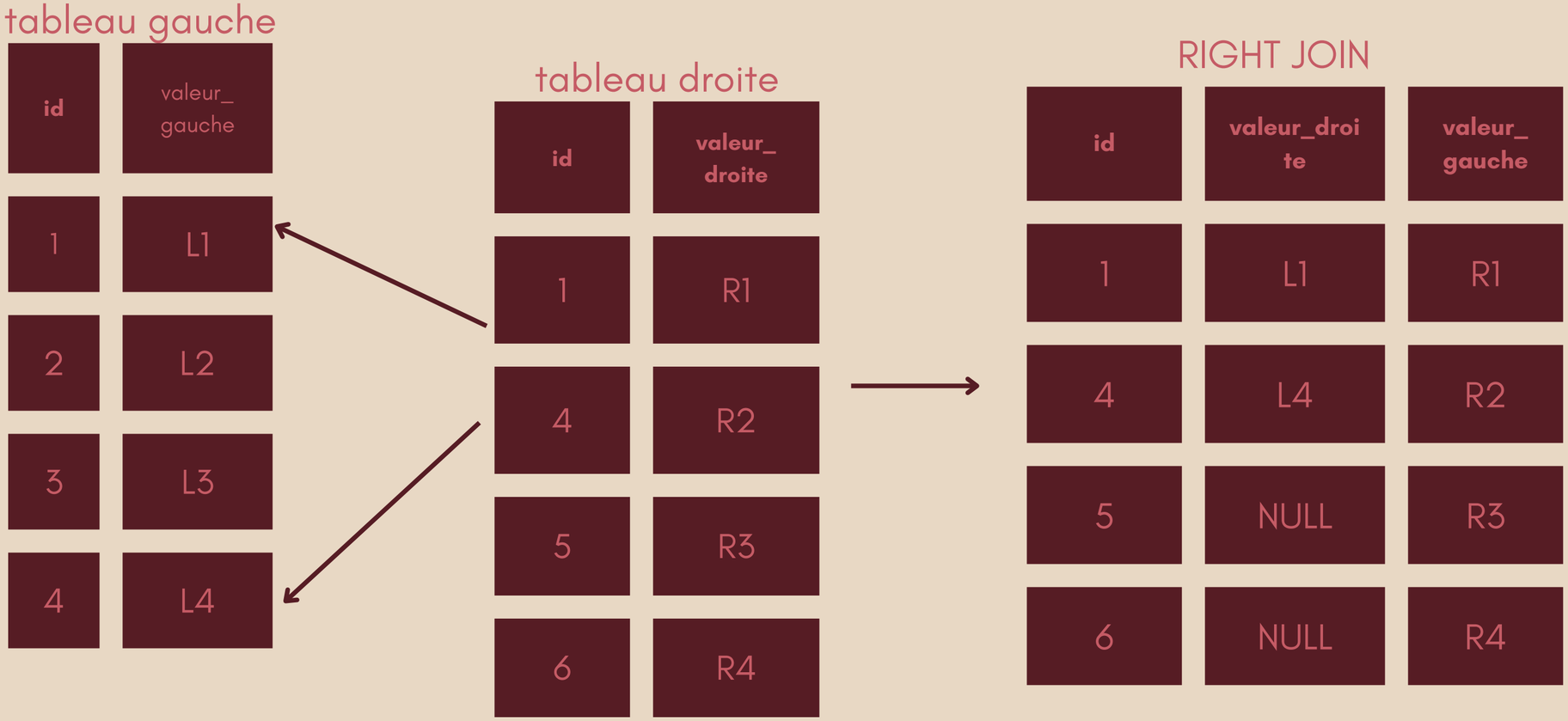
- **Une jointure externe(FULL join) :**
  renvoie toutes les lignes d'un tableau et les lignes correspondantes de l'autre tableau. S'il n'y a pas de correspondance, des valeurs NULL sont renvoyées pour les tableaux dont la ligne ne correspond pas.



- **Une jointure gauche(left join):**
  inclut tous les tableaux du tableau de gauche et les tableaux correspondants du tableau de droite. Si aucune correspondance n'est trouvée, des valeurs NULL sont renvoyées pour les tableaux de droite.



- **une jointure à droite(right join):**
  inclut tous les tableaux de la table de droite, avec les tableaux correspondants de la table de gauche, en remplissant les NULL lorsqu'il n'y a pas de correspondance.



B- Indexer les colonnes utilisées dans les jointures:

Les colonnes utilisées dans les conditions de jointure doivent idéalement être indexées. Sans index, le moteur SQL doit parcourir toute la table (Full Table Scan), ce qui devient très coûteux lorsque les tables contiennent beaucoup de lignes.

C- Réduire le volume de données avant la jointure :

Il est recommandé de filtrer les données avant d'effectuer la jointure, afin de réduire le nombre de lignes traitées.

D. Éviter les jointures inutiles :

Certaines requêtes contiennent des jointures qui ne sont pas réellement nécessaires. Chaque jointure supplémentaire augmente la complexité et le temps d'exécution.

E. Ordre des tables dans les jointures :

Dans certains systèmes de gestion de bases de données, l'ordre des tables peut influencer le plan d'exécution. Il est souvent préférable de commencer par la table la plus petite ou la table la plus filtrée, afin de réduire le nombre de lignes à traiter lors de la jointure.

4- Gestion des transactions:

A- Definition:

- La gestion des transactions est un élément clé de l'optimisation des requêtes SQL. Elle influence à la fois les performances et l'intégrité des données. Une transaction regroupe plusieurs opérations SQL en une unité logique : soit elles sont toutes validées, soit aucune n'est appliquée.

- **Avantages :**
- Annule uniquement les opérations erronées, pas toute la transaction
- Permet de gérer des transactions plus longues tout en gardant le contrôle
- Réduit les blocages et améliore les performances

Une transaction est une séquence d'instructions SQL exécutée de manière indivisible. Les commandes principales sont :

- START TRANSACTION ou BEGIN → démarre la transaction
- COMMIT → valide définitivement toutes les modifications
- ROLLBACK → annule toutes les modifications depuis le début
- SAVEPOINT nom_point → créer un point de sauvegarde
- ROLLBACK TO SAVEPOINT nom_point → annuler uniquement les opérations après ce point

Exemple :

transfert d’argent entre comptes

```sql
START TRANSACTION;
UPDATE comptes SET solde = solde - 100 WHERE id = 1;
UPDATE comptes SET solde = solde + 100 WHERE id = 2;
COMMIT;
```

B- I**mpact sur les performances:**

Une mauvaise gestion des transactions peut ralentir le système :

- Les transactions longues bloquent d'autres requêtes (verrous sur tables ou lignes).
- Une transaction avec trop d'opérations consomme beaucoup de ressources. Une gestion efficace réduit les blocages, améliore la concurrence et optimise l'utilisation des ressources.

C- Bonnes pratiques :

- Transactions courtes : éviter d'inclure des traitements longs entre BEGIN et COMMIT.
- Valider rapidement : utiliser COMMIT dès que possible pour libérer les verrous.
- Limiter le nombre d'opérations : diviser les grosses transactions en plusieurs petites.
- Choisir le niveau d'isolation approprié : READ COMMITTED, REPEATABLE READ, SERIALIZABLE. Un niveau élevé augmente la cohérence mais peut réduire les performances.
- Éviter les verrous inutiles : utiliser des index et des conditions précises (WHERE) pour limiter les lignes bloquées.

D- Gestion des erreurs:

- En cas d'erreur, ROLLBACK permet de revenir à l'état initial et de garantir la cohérence des données.

```
START TRANSACTION;
UPDATE produits SET stock = stock - 5 WHERE id_produit =
ROLLBACK; -- annule les modifications
```

E- Gestion des erreurs et SAVEPOINT :

- Pour des transactions complexes, on peut utiliser SAVEPOINT pour annuler partiellement la transaction :

```
START TRANSACTION;
UPDATE produits SET stock = stock - 5 WHERE id_produit = 10;
SAVEPOINT avant_prix;
UPDATE produits SET prix = prix * 1.1 WHERE id_produit = 10;
-- si erreur sur le prix
ROLLBACK TO SAVEPOINT avant_prix;
COMMIT;
```

5- Évitez les classements et les regroupements inutiles:

⚙En tant que praticiens des données, nous aimons que nos données soient ordonnées et regroupées afin d'obtenir plus facilement des informations. Nous utilisons généralement ORDER BY et GROUP BY dans nos requêtes SQL.

⚙ Cependant, ces deux clauses peuvent être coûteuses en termes de calcul, en particulier lorsqu'il s'agit de grands ensembles de données. Lors du tri ou de l'agrégation des données, le moteur de base de données doit souvent effectuer une analyse complète des données, puis les organiser, identifier les groupes et/ou appliquer des fonctions d'agrégation, généralement à l'aide d'algorithmes gourmands en ressources

Pour optimiser les requêtes, nous pouvons suivre certains des conseils suivants :

- **Minimiser le tri**. Nous ne devrions utiliser ORDER BY qu'en cas de nécessité. Si le tri n'est pas essentiel, l'omission de cette clause peut nous aider à réduire considérablement le temps de traitement.
- **Utilisez des index.** Dans la mesure du possible, nous devons nous assurer que les colonnes impliquées dans ORDER BY et GROUP BY sont indexées .

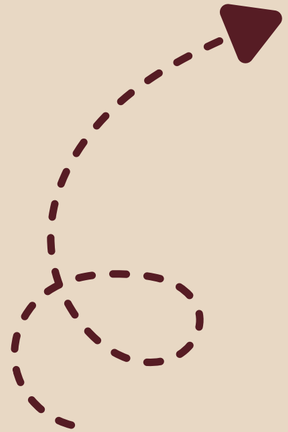
Données pré-agrégées. Pour les requêtes complexes impliquant GROUP BY, nous pourrions pré-agréger les données à un stade antérieur ou dans une vue matérialisée, de sorte que la base de données n'ait pas besoin de calculer les mêmes agrégats à plusieurs reprises.

6-Analyse et monitoring

- Analyser et surveiller les requêtes SQL est essentiel pour maintenir une base de données performante. Même avec une structure correcte, certaines requêtes peuvent ralentir si elles parcourent trop de lignes ou n'utilisent pas efficacement les index. Le monitoring permet d'identifier ces points faibles et d'appliquer les optimisations nécessaires.
- PostgreSQL offre des outils comme EXPLAIN et EXPLAIN ANALYZE pour visualiser le plan d'exécution des requêtes et mesurer leur temps réel. Il est également possible de consulter les statistiques internes ou de suivre les requêtes lentes via les logs. Ces informations aident à comprendre comment les données sont traitées et où se trouvent les goulots d'étranglement (les points qui ralentissent un processus) .

```
exemple=# EXPLAIN ANALYSE SELECT * FROM clients WHERE ville='Paris';
                                                    exemple
                                                  QUERY PLAN

-------------------------------------------------------------------------------------------
----------------------
 Seq Scan on clients  (cost=0.00..1985.00 rows=33603 width=26) (actual time=0.049..14.401 r
ows=33333.00 loops=1)
   Filter: (ville = 'Paris'::text)
   Rows Removed by Filter: 66667
   Buffers: shared hit=735
 Planning:
   Buffers: shared hit=23 dirtied=1
 Planning Time: 1.755 ms
 Execution Time: 16.645 ms
(8 rows)
```



Sequential Scan = lecture complète de la table, ligne par ligne.

Dans cette requête, PostgreSQL utilise un Sequential Scan, ce qui signifie qu'il parcourt toute la table clients ligne par ligne pour vérifier la condition ville = 'Paris'.

Le filtre est appliqué après la lecture des données, ce qui entraîne l'élimination d'un grand nombre de lignes non concernées. Cela montre que la requête traite beaucoup de données inutiles avant de garder les résultats utiles.

Ce type d'exécution est coûteux en performance, car il implique une lecture complète de la table et un temps d'exécution plus élevé.

PostgreSQL lit l'index idx_clients_ville
-Il récupère les positions des lignes correspondant à 'Paris'
-Il construit une "bitmap" (une liste optimisée des blocs à lire)

```
exemple=# CREATE INDEX idx_clients_ville ON clients(ville);
CREATE INDEX
exemple=# EXPLAIN ANALYSE SELECT* FROM clients WHERE ville='Paris';
                                                                 exemple
                                                               QUERY PLAN

-------------------------------------------------------------------------------------------------
-------------------------------------------
 Bitmap Heap Scan on clients  (cost=380.72..1535.75 rows=33603 width=26) (actual time=1.183
..6.857 rows=33333.00 loops=1)
   Recheck Cond: (ville = 'Paris'::text)
   Heap Blocks: exact=735
   Buffers: shared hit=735 read=30
   ->  Bitmap Index Scan on idx_clients_ville  (cost=0.00..372.31 rows=33603 width=0) (actu
al time=1.067..1.067 rows=33333.00 loops=1)
         Index Cond: (ville = 'Paris'::text)
         Index Searches: 1
         Buffers: shared read=30
 Planning:
   Buffers: shared hit=20 read=1
 Planning Time: 2.043 ms
 Execution Time: 9.188 ms
(12 rows)
```

PostgreSQL va lire seulement les blocs nécessaires dans la table,Pas toute la table

PostgreSQL va lire seulement les blocs nécessaires dans la table,Pas toute la table

Types d’accès aux données dans EXPLAIN ANALYZE :

PostgreSQL utilise plusieurs méthodes d’accès aux données, visibles dans EXPLAIN ANALYZE, pour exécuter une requête de manière optimale.

Le Sequential Scan (Seq Scan) correspond à une lecture complète de la table, ligne par ligne. Cette méthode est simple mais devient coûteuse lorsque la table est volumineuse, car toutes les données sont parcourues même si seule une petite partie est nécessaire.

L’Index Scan permet d’utiliser un index pour accéder directement aux lignes recherchées. Cette méthode est plus rapide, car elle évite de lire toute la table, et elle est particulièrement efficace lorsque la requête retourne peu de résultats.

⚙️Le Bitmap Index Scan est utilisé lorsque la requête retourne un nombre important de lignes. PostgreSQL commence par parcourir l'index pour identifier toutes les positions des lignes correspondantes, puis il crée une structure appelée bitmap (une sorte de carte indiquant quels blocs de données doivent être lus). Ensuite, le Bitmap Heap Scan utilise cette carte pour lire uniquement les blocs nécessaires dans la table, ce qui réduit les accès disque et améliore les performances.

⚙️Ainsi, le choix entre ces méthodes dépend du volume de données à traiter : un scan séquentiel pour les petits volumes, un index scan pour des recherches ciblées, et un bitmap scan pour des volumes intermédiaires à importants.

Analyse des problèmes avec EXPLAIN ANALYZE:

L'analyse du plan d'exécution permet d'identifier plusieurs types de problèmes pouvant affecter les performances d'une requête.

-Un premier problème fréquent est l'utilisation d'un Sequential Scan sur une grande table. Cela signifie que PostgreSQL parcourt toutes les lignes, ce qui augmente fortement le temps d'exécution.

-Un autre problème est le traitement inutile de données, visible lorsque un grand nombre de lignes est supprimé après filtrage. Cela indique que la requête lit beaucoup de données qui ne sont finalement pas utilisées.

-L'absence d'index sur les colonnes utilisées dans les conditions (WHERE) constitue également une cause importante de ralentissement. Sans index, le système ne peut pas accéder directement aux données recherchées.

-Un problème peut aussi apparaître lorsque le moteur choisit une méthode moins efficace, comme un scan séquentiel au lieu d'un accès par index. Cela peut être dû à des statistiques incorrectes ou à une mauvaise estimation du nombre de lignes.

-Enfin, un temps d'exécution élevé dans certaines opérations indique la présence de goulots d'étranglement, c'est-à-dire des étapes qui ralentissent l'ensemble de la requête.

Conclusion

- L'optimisation des requêtes SQL est une étape essentielle pour garantir de bonnes performances dans une base de données. Elle permet de réduire le temps d'exécution des requêtes, d'améliorer la rapidité d'accès aux données et d'utiliser plus efficacement les ressources du système.



- Pour obtenir de meilleures performances, il est important d'adopter de bonnes pratiques lors de l'écriture des requêtes, comme sélectionner uniquement les colonnes nécessaires, utiliser correctement les filtres et éviter les opérations inutiles. L'utilisation appropriée des index, l'optimisation des jointures et une bonne gestion des transactions jouent également un rôle important dans l'amélioration des performances.

- Enfin, l'analyse régulière des requêtes à l'aide d'outils comme EXPLAIN permet d'identifier les requêtes lentes et d'apporter les améliorations nécessaires. Ainsi, tester, analyser et optimiser les requêtes de manière continue est indispensable pour maintenir un système de base de données efficace et performant.

THANK YOU